



Universidade Estadual de Mato Grosso do Sul

Ciência da Computação

Banco de Dados

SQL - parte 2

Prof. Nilton

nilton@comp.uems.br

Sintaxe do comando SELECT

```
SELECT <lista de campos>  
FROM <lista de tabelas>  
[WHERE <condição>]  
[GROUP BY <campos que definem o grupo>  
[HAVING <condição sobre a função de grupo>] ]  
[ORDER BY <campos do resultado da consulta>]
```

Comparação de substrings

O operador LIKE permite a comparação com partes de uma string usando coringas “%” ou “_”.

```
SELECT * FROM Empregado WHERE ender LIKE "R X"
```

- R X

```
SELECT * FROM Empregado WHERE ender LIKE "%da Silva"
```

- Antonieta da Silva, da Silva

```
SELECT * FROM Empregado WHERE ender LIKE "C%"
```

- Ceará, Comendador

```
SELECT * FROM Empregado WHERE ender LIKE "%da%"
```

- Manoel da Costa, da Costa e Silva, R Hilda

Comparação de substrings

O operador LIKE permite a comparação com partes de uma string usando coringas “%” ou “_”.

```
SELECT * FROM Empregado WHERE ender LIKE "Star ----"
```

- Star Wars, Star Trek

```
SELECT * FROM Empregado WHERE ender LIKE "Star ---- %"
```

- Star Wars 2018

```
SELECT * FROM Empregado WHERE ender LIKE "%rly%" || "%ill%"
```

- Beverly Hills Cop, Praias de Beverly Hills

Operadores de conjunto – UNION (união), EXCEPT (diferença), INTERSECT (interseção)

Primeiro nome e número de projeto que tenha um Silva como último nome que trabalha no projeto ou é chefe do departamento que controla o projeto''

$$\text{Prj}(\text{nproj}, \text{numdep}) \leftarrow \pi_{\text{numproj}, \text{ndep}} (\text{Proj})$$
$$\pi_{\text{nominic}, \text{nproj}} (\sigma_{\text{nomefim}='Silva'} (\text{Empreg}) \bowtie_{\text{numat}=\text{numemp}} \text{Trab})) \cup \pi_{\text{nominic}, \text{nproj}} (\text{Prj} \bowtie \text{Dep} \bowtie_{\text{nchefe}=\text{numat}} (\sigma_{\text{nomefim}='Silva'} (\text{Empreg})))$$

```
(SELECT nominic, nproj AS numproj
FROM   TrabalhaEm, Empregado
WHERE  numemp = numat AND nomefim = 'Silva')
UNION
(SELECT nominic, numproj
FROM   Projeto, Departamento, Empregado
WHERE  ndep = numdep AND nchefe = numat AND
nomefim = 'Silva');
```

Consultas aninhadas ou subconsultas

Forma alternativa de especificar consultas envolvendo relacionamentos entre tabelas.

Otimização

Filtragens prévias de dados na subconsulta

Apenas tuplas/atributos de interesse são combinados com dados da(s) tabela(s) da consulta externa

Cláusulas

- nome_atributo [NOT] IN (consulta_SQL)
- nome_atributo [< | <= | > | >= | <> | !=] ANY (consulta_SQL)
- nome_atributo [< | <= | > | >= | <> | !=] ALL (consulta_SQL)
- nome_atributo [NOT] EXISTS (consulta_SQL)

Subconsultas que produzem valor escalar

A subconsulta produz apenas um elemento como resultado, para comparação de elemento-elemento.

```
select lista_atributos
from tabela1 [, ...]
where atributo_ou_expressão
    [=|<|<=|>|>=|<>|!=] (consulta_SQL com um
    único elemento)
```

Relacione os funcionários que têm salário maior que o funcionário de CPF 999.999.999-99.

```
SELECT nome
FROM    Funcionario
WHERE   salario > (
        SELECT salario
        FROM    Funcionario
        WHERE   cpf = 999999999999
    );
```

O operador IN

Compara um valor v com um conjunto (ou multiconjunto) de valores V e avalia como verdadeiro se v for um dos elementos de V. Caso contrário, avalia falso.

Para comparar elemento-conjunto. Devem ser compatíveis.

```
select lista_atributos  
from tabela1 [...]  
where atributo_ou_expressão [NOT] IN  
  (consulta_SQL)
```


Subconsultas - IN

Relacione os códigos das locações e os nomes dos clientes que as fizeram para contas de locação que estão vencendo hoje

```
SELECT idloc, nomecli
FROM   Cliente NATURAL JOIN Locacao
WHERE  idct IN (
        SELECT idct
        FROM   Conta
        WHERE  dtvenc = SYS_DATE()
      ) ;
```

Nomes dos empregados que não têm dependentes

$$\pi_{\text{nominic, nomefim}}(\text{Empreg}) - \pi_{\text{nominic, nomefim}}(\text{Empreg} \bowtie_{\text{numat=numemp}} \text{Depen})$$

```
SELECT nominic, nomefim
FROM Empregado
WHERE numat not IN (
    SELECT numemp
    FROM Dependente
);
```

Qual o resultado do seguinte SELECT ?

```
SELECT DISTINCT numemp
FROM TrabalhaEm
WHERE (nproj, horas) IN (
    SELECT nproj, horas
    FROM TrabalhaEm
    WHERE numemp = '1234'
);
```

Subconsultas – ANY

Para comparação de elemento-conjunto.

Testa se um valor é >, <, =, ... que **algum** valor de um conjunto.

```
select lista_atributos  
from tabela1 [, ...]  
where atributo_ou_expressão [=|<|<=|>|>=|<>|!=] ANY  
  (consulta_SQL)
```

Subconsultas – ANY

Encontrar nomes dos empregados cuja data de nascimento é posterior à de algum empregado do departamento 5

```
SELECT  nominic, nomint, nomefim
FROM    Empregado
WHERE    dtnasc > ANY (
        SELECT  dtnasc
        FROM    Empregado
        WHERE    ndepto = 5
    ) ;
```

Subconsultas – ALL

Para comparação de elemento-conjunto.

Testa se um valor é >, <, =, ... que **todos** valores de um conjunto.

```
select lista_atributos  
from tabela1 [, ...]  
where atributo_ou_expressão [=|<|<=|>|>=|<>|!=]  
      ALL(consulta_SQL)
```

Subconsultas – ALL

Encontrar nomes dos clientes com data de admissão máxima

```
SELECT nomecli
FROM   Cliente
WHERE  dtadm >= ALL (
        SELECT dtadm
        FROM   Cliente
      ) ;
```

**Nome dos empregados cujo salário é maior do
que o salário de todos os empregados do
departamento 5**

```
SELECT nominic, nomint, nomefim  
FROM Empregado  
WHERE salario > ALL (  
    SELECT salario  
    FROM Empregado  
    WHERE ndepto = 5  
) ;
```


EXISTS

Quantificador existencial do cálculo relacional

- Testa se um predicado é V ou F na subconsulta
- Para cada tupla da consulta externa a ser analisada, a subconsulta é executada (consulta correlacionada).

Se EXISTS retornar verdadeiro - não vazia. Se EXISTS retornar falso - vazia

```
select lista_atributos  
from tabela1 [, ...]  
where [NOT] EXISTS (consulta_SQL)
```

Subconsultas - EXISTS

Relacione os nomes dos clientes que estão em atraso na devolução das fitas que locaram.

```
SELECT nomecli
FROM   Cliente
WHERE  EXISTS (
        SELECT *
        FROM   Locacao
        WHERE  idcli = Cliente.idcli
        AND dtdevolprev < SYS_DATE()
        AND dtdevolreal IS NULL
      ) ;
```

Nomes dos chefes de departamento que têm pelo menos um dependente

$\pi_{\text{nominic, nomefim}} (\text{Dep} \bowtie_{\text{nchefe=numat}} (\text{Empreg} \bowtie_{\text{numat=numemp}} \text{Depen}))$

```
SELECT  nominic, nomefim
FROM    Empregado
WHERE   EXISTS (
        SELECT *
        FROM   Dependente
        WHERE  numat = numemp)
AND     EXISTS (
        SELECT *
        FROM   Departamento
        WHERE  numat = nchefe) ;
```

Nomes dos empregados que não têm dependentes

$\pi_{\text{nominic, nomefim}}(\text{Empreg}) - \pi_{\text{nominic, nomefim}}(\text{Empreg} \bowtie_{\text{numat=numemp}} \text{Depen})$

```
SELECT nominic, nomefim
FROM Empregado
WHERE NOT EXISTS (
    SELECT *
    FROM Dependente
    WHERE numat = numemp
);
```

**Nome dos empregados que trabalham em
projetos controlados pelo mesmo departamento
em que estão lotados**

```
SELECT e.nominic, e.nomint, e.nomefim  
FROM Empregado AS e  
WHERE EXISTS (  
    SELECT p.*  
    FROM Projeto AS p, TrabalhaEm AS t  
    WHERE p.ndep=e.ndepto AND t.numemp=  
    e.numat AND t.nproj = p.numproj  
);
```

A função UNIQUE(Q)

Função booleana

Parâmetro: consulta em SQL

Retorna Verdadeiro se duplicidade de tuplas não são encontradas como resultado da consulta Q

Empregados que trabalham em apenas um projeto

```
SELECT *  
FROM Empregado AS e  
WHERE UNIQUE (  
    SELECT t.numemp  
    FROM TrabalhaEm AS t  
    WHERE e.numat = t.numemp  
);
```

Funções Agregadas

SQL possui as seguintes funções agregadas:
SUM, COUNT, MIN, MAX e AVG

COUNT

Conta o número de tuplas resultante da consulta.

SUM, MIN, MAX e AVG

aplicadas sobre um conjunto de valores

Soma de todos os salários dos empregados

$$\gamma_{\text{SUM}(\text{sal}) \rightarrow \text{sum_sal}}(\text{Empreg})$$

```
SELECT    SUM(sal)
FROM      Empreg
```

Número de empleados

$$\gamma_{\text{COUNT}(*)\rightarrow \text{ct_emp}}(\text{Empreg})$$

```
SELECT  
FROM
```

```
COUNT (*)  
Empreg
```

Maior salário entre os empregados

$$\gamma_{\text{MAX}(\text{sal}) \rightarrow \text{max_sal}}(\text{Empreg})$$

```
SELECT    MAX (sal)
FROM      Empreg
```

**Considerando todos os empregados, selecionar
a soma de todos os salários, o maior e menor
salários e a média dos salários**

```
SELECT  SUM(salario) ,  
        MAX(salario) ,  
        MIN(salario) ,  
        AVG(salario)  
FROM      Empregado ;
```

Média dos salários dos empregados do Departamento de Pesquisa

```
SELECT AVG(salario)
FROM    Empregado, Departamento
WHERE   nomedep = 'Pesquisa' AND
        numdep = ndepto
```

Número de empregados do Departamento de Pesquisa

```
SELECT COUNT (*)  
FROM Empregado, Departamento  
WHERE nomedep = 'Pesquisa' and  
numdep = ndepto;
```

Número de diferentes salários da empresa

```
SELECT COUNT(DISTINCT salario)
FROM Empregado;
```

Agrupamento

Em alguns casos necessitamos aplicar a função agregada em subgrupos de linhas

Média dos salários dos empregados por departamento

$\gamma_{ndepto, AVG(sal) \rightarrow avg_sal} (Empreg)$

```
SELECT    ndepto, AVG(sal)
FROM      Empreg
GROUP BY  ndepto;
```

Maior salário de um empregado por departamento

$\gamma_{ndepto, \text{MAX}(\text{sal}) \rightarrow \text{max_sal}}(\text{Empreg})$

```
SELECT    ndepto, MAX(sal)
FROM      Empreg
GROUP BY  ndepto
```

Número de salário iguais dos empregados

$$\gamma_{\text{sal}, \text{COUNT}(*)} \rightarrow \text{ct_sal}(\text{Empreg})$$

```
SELECT    sal, COUNT(*)  
FROM      Empreg  
GROUP BY  sal
```

A média dos salários e o número de empregados para cada departamento

```
SELECT ndepto, COUNT(*) ,  
        AVG(salario)  
FROM    Empregado  
GROUP BY ndepto;
```

Para cada projeto o seu número, o seu nome e a quantidade de empregados que trabalham no projeto

```
SELECT numproj, nomeproj, COUNT(*)  
FROM   TrabalhaEm, Projeto  
WHERE  nproj = numproj  
GROUP BY numproj, nomeproj;
```

Empregados que possuem mais de 2 dependentes

```
SELECT *  
FROM Empregado  
WHERE ( SELECT COUNT (*)  
        FROM Dependente  
        WHERE numat = numemp)  
      > 2 ;
```

Para cada projeto com mais de 5 empregados,
buscar o número do projeto, o nome do projeto
e o número de empregados que trabalham nele

```
SELECT numproj, nomeproj, COUNT(*)  
FROM    Projeto, TrabalhaEm  
WHERE   numproj = nproj  
GROUP BY numproj, nomeproj  
HAVING COUNT(*) > 5;
```

Junções

→ **CROSS JOIN**

→ **NATURAL JOIN**

→ **INNER JOIN ou JOIN**

→ **OUTER JOIN**

→ **LEFT OUTER JOIN ou LEFT JOIN**

→ **RIGHT OUTER JOIN ou RIGHT JOIN**

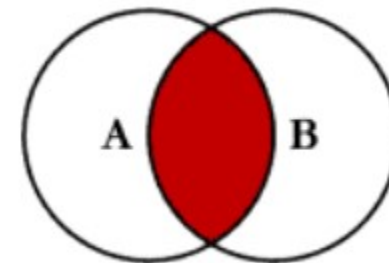
→ **FULL OUTER JOIN ou FULL JOIN**

CROSS JOIN

Simplesmente substitui a vírgula nas operações de junção (FROM). Se não houver condição de junção o resultado é produto cartesiano.

Relacione os nomes dos empregados e os nomes de seus dependentes.

```
SELECT e.nominic, d.depend  
FROM  Empregado AS e CROSS JOIN  
      Dependente AS d  
WHERE e.numat = d.numemp
```

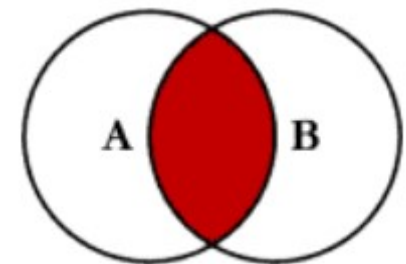


NATURAL JOIN

Serve para indicar uma junção natural, logo não é necessário indicar quais as colunas de junção. Elimina a duplicidade dos atributos de junção.

Relacione o nome e o endereço dos empregados que trabalham no departamento de informática.

```
SELECT nominic, ender  
FROM   Empregado NATURAL JOIN  
       Departamento AS Depto(nomedep,  
                             ndepto, nchefe, dtinchef)  
WHERE  nomedep = 'Informática'
```



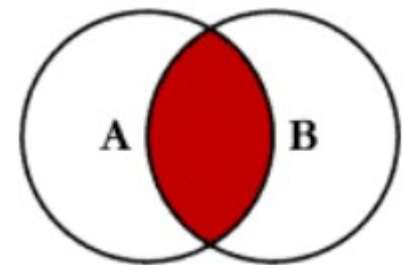
INNER JOIN ou JOIN

Forma mais comum de junção em que são especificadas as condições de junção usando a cláusula ON. Retorna as tuplas que satisfaçam as condições de junção.

Relacione os nomes dos empregados e os nomes de seus dependentes.

```
SELECT nominic, depend  
FROM Empregado INNER JOIN Dependente  
ON numat = numemp
```

```
SELECT nominic, depend  
FROM Empregado JOIN Dependente  
ON numat = numemp
```

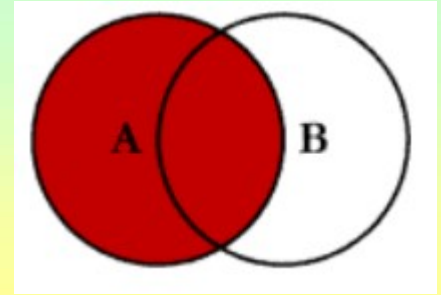


OUTER JOIN

Uma junção em que as tuplas de uma das relações aparecem no resultado mesmo que não tenham par na outra relação (com os atributos da outra relação preenchidos com valores nulos).

- ◆ **LEFT OUTER JOIN:** As tuplas da relação do lado esquerdo aparecem mesmo não existindo correlação;
- ◆ **RIGHT OUTER JOIN:** As tuplas da relação do lado direito aparecem mesmo não existindo correlação;
- ◆ **FULL OUTER JOIN:** As tuplas de ambas relações aparecem mesmo não existindo correlação;

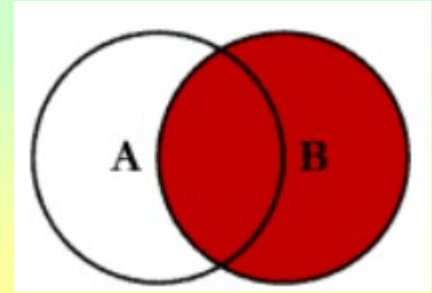
LEFT OUTER JOIN



Buscar o nome de todos os empregados. Se o empregado possuir dependentes, selecionar o nome e sexo de cada um dos seus dependentes

```
SELECT e.nominic, d.depend, d.sexo  
FROM   Empregado AS e LEFT OUTER JOIN  
       Dependente AS d ON e.numat = d.numemp
```

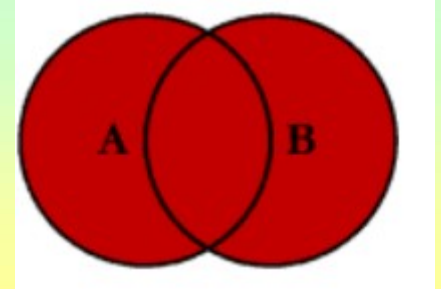
RIGHT OUTER JOIN



**Buscar os departamentos e seus respectivos chefes,
incluindo os departamentos que não tem chefe.**

```
SELECT d.nomedep, e.nominic  
FROM   Empregado AS e RIGHT OUTER JOIN  
       Departamento AS d ON e.numat = d.nchefe
```

FULL OUTER JOIN



Buscar os departamentos e seus respectivos chefes, incluindo os departamentos que não tem chefe e os empregados que não são chefes de nenhum departamento.

```
SELECT d.nomedep, e.nominic  
FROM   Empregado AS e FULL OUTER JOIN  
       Departamento AS d ON e.numat = d.ncheefe
```

Nomes dos empregados e nomes de seus supervisores diretos

$\text{Emp}(\text{empinic}, \text{empfim}, \text{nsup}) \leftarrow \pi_{\text{nominic}, \text{nomefim}, \text{nsup}} (\text{Empreg})$

$\text{Sup}(\text{supinic}, \text{supfim}, \text{nsup}) \leftarrow \pi_{\text{nominic}, \text{nomefim}, \text{numat}} (\text{Empreg})$

$\pi_{\text{empinic}, \text{empfim}, \text{supinic}, \text{supfim}} (\text{Emp} \bowtie \text{Sup})$

```
SELECT einic, efim, sinic, sfim
FROM   Empregado(nominic, nomefim, nsuper) AS
      Emp(einic, efim, nsup) NATURAL JOIN
      Empregado(nominic, nomefim, numat) AS
      Sup(sinic, sfim, nsup);
```


Recuperar o nome do empregado e, se o empregado possuir supervisor, recuperar também o nome do seu supervisor imediato

```
SELECT e.nominic as IniEmp,  
       e.nomefim as FimEmp,  
       s.nominic as IniSup,  
       s.nomefim as FimSup  
FROM   Empregado e LEFT OUTER JOIN  
       Empregado s  
ON     e.nsuper = s.numat;
```

Para cada projeto localizado em Dourados
selecionar o número do projeto, o número e o nome
do chefe do departamento que controla o projeto

```
SELECT numproj, ndep, nominic  
FROM    Projeto JOIN  
        Departamento ON numdep=ndep  
        JOIN Empregado ON nchefe=numat  
WHERE   locproj = 'Dourados'
```

Views em SQL

- É uma tabela derivada de outras tabelas
- A derivação a partir de tabelas de base (física) ou outras *views* (virtuais)
- Útil em consultas com bastante frequência
- Operações SQL
 - Atualização: única tabela (atualizável), varias tabelas (as vezes atualizável), funções de agregação (não são atualizáveis)

Views em SQL

```
CREATE VIEW <nome_visão> AS  
<consulta SQL>
```

Empregado(nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsuper, ndepto)

TrabalhaEm (numemp, nproj, horas)

Projeto (nomeproj, numproj, locproj, ndep)

Projetos alocados aos empregados.

```
CREATE VIEW V_PROJ_EMPR AS  
SELECT *  
FROM    Empregado, TrabalhaEm, Projeto  
WHERE   numat=numemp AND nproj = numproj;
```

Views em SQL

Empregado(nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsuper, ndepto)

TrabalhaEm (numemp, nproj, horas)

Projeto (nomeproj, numproj, locproj, ndep)

Projetos alocados aos empregados.

```
CREATE VIEW V_PROJ_EMPR(nominic, nomeproj, horas) AS
SELECT nominic, nomeproj, horas
FROM    Empregado, TrabalhaEm, Projeto
WHERE   numat=numemp AND nproj = numproj;
```

Link sobre SQL

<https://www.w3schools.com/sql/default.asp>